



MNISTCNN-2_26.ipynb

Act as an expert Senior Python & Machine Learning Instructor. I have attached a Jupyter Notebook (`.ipynb`)

I do not want high-level summaries or generic documentation. Instead, please provide an exhaustive, line-by-line, mechanical breakdown of every single operational line of code in this file.

For EVERY line of code (especially the mathematical steps, array manipulations, and plotting configurations), please address the following structural points:

1. UNDERLYING MECHANICS & ARGUMENTS: What do the specific function parameters or arguments mean at a low level? (For example, if a line reads `axis=0`, `full_matrices=False`, or `::-1`, explain exactly what that specific configuration tells the computer to do step-by-step).
2. STRUCTURAL & SHAPE TRANSFORMATION: What are the input array shapes and data types before this line executes, and what is the exact output shape/data type immediately after it executes? (Explicitly use matrix dimensions like (N, D), (D, D), etc., to show how data is being stretched, collapsed, or transformed).
3. BEHAVIORAL ALTERNATIVES: What would happen to the code's behavior, execution flow, data shapes, and logic if those specific parameters were changed to other valid/invalid states? (Provide counter-examples of alternative parameter values and explain why they would either alter the outcome or break the script).
4. ALGORITHMIC & MATH JUSTIFICATION: Why is this exact configuration mathematically or structurally mandatory for the algorithm to function properly?

Please maintain this deep, parameter-focused engineering perspective throughout your entire line-by-line response.

Code Cell 1: Data Acquisition and Preprocessing

Line 1: `import tensorflow as tf`

- **Underlying Mechanics & Arguments:** This instruction invokes Python's underlying import system to locate, load, and initialize the `tensorflow` package, binding its top-level namespace to the reference name `tf`. Mechanically, it parses the package modules, builds its structural graph dependencies, maps dynamic C++ bindings via the Java Native Interface/Pybind11 layer, and establishes global state managers like the eager execution runtime engine and memory allocators for available hardware backends (CPU/GPU/TPU).
 - **Structural & Shape Transformation:** * *Input:* None (System state pre-execution).
 - *Output:* A Python module object referenced by `tf`. No multi-dimensional arrays or standard mathematical matrices are instantiated or altered.
 - **Behavioral Alternatives:** Replacing this with `import tensorflow` without an alias requires modifying subsequent invocations to use the full namespace `tensorflow.keras...`. Altering it to a non-existent package like `import tensorflow_gpu` will raise a runtime `ModuleNotFoundError`.
 - **Algorithmic & Math Justification:** Mandatory for establishing the core execution graph environment. It acts as the orchestrator for all downstream computational components, tensor allocations, and differentiation mechanics.
-

Line 2: `from tensorflow.keras import layers, models`

- **Underlying Mechanics & Arguments:** This statement queries the initialized `tf` module tree, drills down into the Keras API submodule (`tensorflow.keras`), and extracts explicit references to the subpackages `layers` and `models`. By injecting these specific modules directly into the local namespace, it avoids the execution overhead and visual noise of deeply nested namespace lookups during neural network structural construction.
- **Structural & Shape Transformation:** * *Input:* Existing `tf` module framework in memory.
 - *Output:* Two localized, executable package namespaces: `layers` and `models`. No array transformations occur.
- **Behavioral Alternatives:** Omitting this line would require re-writing layer instantiation syntax as `tf.keras.layers.Conv2D`, which functions identically but increases structural verbosity. Typos in module references (e.g., `import layer`) throw an explicit `ImportError`.

- **Algorithmic & Math Justification:** Structurally necessary to provide high-level abstractions for deep neural network configurations without manually writing low-level weight matrices, bias vectors, and custom backward propagation loops.
-

Line 3: `import matplotlib.pyplot as plt`

- **Underlying Mechanics & Arguments:** This loads the `pyplot` state-machine interface from the `matplotlib` visualization suite, aliasing it locally to `plt`. This initializes an internal backend renderer that monitors figure creation, axis positioning, color maps, and canvas state stacks.
 - **Structural & Shape Transformation:** * *Input:* System configuration state.
 - *Output:* A module reference `plt` monitoring a clean visualization state stack. No tensor transformations occur.
 - **Behavioral Alternatives:** Using `import matplotlib` requires accessing methods via `matplotlib.pyplot`. Changing the backend renderer (e.g., `matplotlib.use('Agg')`) prior to this call will alter how images are displayed or saved to disk.
 - **Algorithmic & Math Justification:** Necessary for evaluating model output. It provides the visual layout mechanics required to map 2D image matrices back into standard continuous spatial graphs for manual diagnostic inspection.
-

Line 4: `import numpy as np`

- **Underlying Mechanics & Arguments:** Loads the fundamental NumPy (Numerical Python) package, binding its C-implemented multi-dimensional array structures (`ndarray`) and mathematical operation arrays under the `np` alias. It sets up contiguous memory allocation routines, vectorized operations pointers (vectorized SIMD instructions), and locks hardware linear algebra routines (BLAS/LAPACK implementations).
 - **Structural & Shape Transformation:** * *Input:* Global workspace state.
 - *Output:* Global namespace mapping for `np`. No structural data transformation.
 - **Behavioral Alternatives:** Using a native Python list structure instead of NumPy data arrays drops SIMD acceleration, increasing the computational execution complexity of element-wise transformations from $O(N)$ native C-speed to high-overhead Python looping.
 - **Algorithmic & Math Justification:** Mandatory for linear mathematical mutations outside the specialized TensorFlow graph wrapper, specifically for high-performance matrix restructuring, indexing, and argmax reductions.
-

Line 5: `fashion_mnist = tf.keras.datasets.fashion_mnist`

- **Underlying Mechanics & Arguments:** References the exact storage module containing the Fashion-MNIST dataset wrapper within Keras's local collection. It does not load data into RAM at this precise instant; it merely registers an abstract API dataset object pointer pointing to the dataset's standard remote storage URLs or its cached binary location in the host system directory.
 - **Structural & Shape Transformation:** * *Input:* Pre-existing `tf` object properties.
 - *Output:* An instance of the Keras dataset provider helper class assigned to `fashion_mnist`.
 - **Behavioral Alternatives:** Pointing to `tf.keras.datasets.mnist` instead would change the targets from structural clothing elements to handwritten digits, changing the entire feature distribution from complex multi-texture images to simple high-contrast shapes.
 - **Algorithmic & Math Justification:** This step isolates the target data stream source, establishing the benchmark dataset framework before executing lower-level binary array extraction.
-

Line 6: `(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()`

- **Underlying Mechanics & Arguments:** This executes the data fetching and decompression loop. It checks the local cache directory (`~/.keras/datasets/`); if absent, it fetches four compressed byte files via HTTPS. Once verified, it unpacks the binary structures, parses the header files to extract tensor dimensions, reads raw data blocks directly into memory, and unpacks them into two structured Python tuples containing decoupled features and targets.
 - **Structural & Shape Transformation:** * *Input:* Abstract class pointer `fashion_mnist`.
 - *Output:* Four highly specific NumPy arrays:
 - `train_images` : Shape `(60000, 28, 28)`, Type: `uint8` (unsigned 8-bit integer, values 0 to 255).
 - `train_labels` : Shape `(60000, )`, Type: `uint8` (integers 0 through 9).
 - `test_images` : Shape `(10000, 28, 28)`, Type: `uint8`.
 - `test_labels` : Shape `(10000, )`, Type: `uint8`.
 - **Behavioral Alternatives:** If network operations fail mid-download, a connectivity runtime error is thrown. If the destination variables are rearranged or unpacking brackets are improperly balanced, a `ValueError: too many values to unpack` occurs due to structure mismatching.
 - **Algorithmic & Math Justification:** Establishes the exact dimensions of the image matrices: $N_{train} = 60,000$ and $N_{test} = 10,000$ continuous matrices, each matching a strict spatial coordinate domain of $H \times W = 28 \times 28$.
-

Line 7:

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

- **Underlying Mechanics & Arguments:** Instantiates a native Python indexed sequence list containing ten distinct text strings. This serves as a human-readable look-up dictionary mapped to the ordinal integers (0 to 9) inside the label tensors.
 - **Structural & Shape Transformation:** * *Input:* None.
 - *Output:* Standard 1D Python string list of length 10.
 - **Behavioral Alternatives:** Modifying the order of these elements breaks the mapping to the dataset targets, causing the model evaluation steps to print incorrect textual category labels.
 - **Algorithmic & Math Justification:** This list does not influence mathematical optimizations or computational loss values. It is purely an evaluation interface that maps index values back to their categorical names.
-

Line 8: `train_images = train_images / 255.0`

- **Underlying Mechanics & Arguments:** This line performs an element-wise array division. NumPy captures the scalar value `255.0` (explicit float) and uses broadcasting mechanics to divide each individual item in the array. This converts the array from integer storage to floating-point representation in memory.
 - **Structural & Shape Transformation:** * *Input:* `train_images` of shape `(60000, 28, 28)` with `uint8` data type.
 - *Output:* `train_images` of shape `(60000, 28, 28)` upscaled to `float64` data type, bounding all active pixel values inside a normalized interval `[0.0, 1.0]`.
 - **Behavioral Alternatives:** Dividing by an integer `255` instead of float `255.0` can cause legacy environments to perform truncating division, mapping all values below 255 directly to 0. Omitting normalization entirely forces raw inputs `([0, 255])` into the weight matrices, causing exploding gradients during backpropagation.
 - **Algorithmic & Math Justification:** Deep neural networks utilize gradient descent optimizations that perform best when feature variables are small and centered. Keeping inputs within a `[0, 1]` range stabilizes the activation functions (like ReLU), preventing large numerical swings during forward passes.
-

Line 9: `test_images = test_images / 255.0`

- **Underlying Mechanics & Arguments:** Identical to the previous transformation. It uses element-wise scalar broadcasting to scale the testing dataset values using the same range configuration.

- **Structural & Shape Transformation:** * *Input:* `test_images` of shape `(10000, 28, 28)` with `uint8` data type.
 - *Output:* `test_images` of shape `(10000, 28, 28)` with `float64` data type, values restricted to `[0.0, 1.0]`.
 - **Behavioral Alternatives:** Normalizing data with a different denominator (e.g., `128.0`) would introduce a massive out-of-distribution feature skew relative to the training dataset, significantly degrading test evaluation accuracy.
 - **Algorithmic & Math Justification:** Data transformations applied to training features must be replicated exactly on evaluation inputs to ensure the mathematical continuity of the model's learned weights.
-

Line 10: `train_images = train_images.reshape((train_images.shape[0], 28, 28, 1))`

- **Underlying Mechanics & Arguments:** Invokes the NumPy `.reshape()` function, which creates a new structural layout view of the array's underlying data. This operation is highly efficient ($O(1)$) because it only updates the metadata stride paths while keeping the original contiguous block of memory unchanged. The function takes a 4-tuple argument specifying the new target tensor structure.
 - **Structural & Shape Transformation:** * *Input:* Shape `(60000, 28, 28)`, representing (Samples, Height, Width).
 - *Output:* Shape `(60000, 28, 28, 1)`, explicitly representing (Samples, Height, Width, Channels). The data type remains `float64`.
 - **Behavioral Alternatives:** Passing an invalid total element count (e.g., `(60000, 28, 28, 2)`) raises a `ValueError` because the requested shape requires double the original number of elements ($60000 \times 28 \times 28 \times 1$).
 - **Algorithmic & Math Justification:** 2D Convolutional layers in Keras mandate a strict 4D tensor rank format: `(Batch, Height, Width, Channels)`. Even though grayscale images possess a single color plane, the channel dimension must be explicitly defined to establish proper mathematical tensor contraction during the convolution steps.
-

Line 11: `test_images = test_images.reshape((test_images.shape[0], 28, 28, 1))`

- **Underlying Mechanics & Arguments:** Performs the exact same spatial stride remapping on the evaluation image array, adding an explicit single-channel axis to match the expected network dimensions.
- **Structural & Shape Transformation:** * *Input:* `test_images` matrix with a 3D shape of `(10000, 28, 28)`.

- **Output:** `test_images` matrix with a 4D shape of $(10000, 28, 28, 1)$.
 - **Behavioral Alternatives:** Omitting this step for the test set while reshaping the training set will lead to a dimension mismatch error during model evaluation, as the network expects a 4D input tensor.
 - **Algorithmic & Math Justification:** Ensures structural compatibility between the test data stream and the spatial processing dimensions defined in the input layer of the convolutional model.
-

Line 12: `print(f"Training data shape: {train_images.shape}")`

- **Underlying Mechanics & Arguments:** Evaluates a Python f-string by accessing the `.shape` attribute tuple of the `train_images` array and outputs the text string directly to the standard output system buffer (`sys.stdout`).
 - **Structural & Shape Transformation:** No modifications are made to the storage matrices or active states.
 - **Behavioral Alternatives:** Modifying or deleting this statement has no impact on execution flow, as it is purely an informative diagnostic tool.
 - **Algorithmic & Math Justification:** Provides visual confirmation that the input tensor dimensions match the structural signature required by the network before starting the initialization steps.
-

Line 13: `print(f"Testing data shape: {test_images.shape}")`

- **Underlying Mechanics & Arguments:** Accesses the `.shape` structural metadata property of the `test_images` array, constructs a formatted text string, and prints it to the output console.
 - **Structural & Shape Transformation:** No data matrices are shifted or mutated.
 - **Behavioral Alternatives:** Removing this line has zero impact on the mathematical execution or convergence behavior of downstream operations.
 - **Algorithmic & Math Justification:** Serves as a manual check to ensure the validation pipeline arrays match the intended dimensions prior to model evaluation.
-

Code Cell 2: Network Topology Architecture & Compilation

Line 14: `model = models.Sequential([`

- **Underlying Mechanics & Arguments:** Instantiates a Keras `Sequential` model object container from the imported `models` namespace. This class creates a linear structural stack where layers are chained sequentially. It initializes a clean network graph, overrides execution trackers, and registers a container array designed to dynamically ingest and connect structural layers.

- **Structural & Shape Transformation:** * *Input:* Non-initialized object space.
 - *Output:* An empty Keras `Sequential` layer container model reference.
 - **Behavioral Alternatives:** Utilizing Keras Functional API syntax (`keras.Model(inputs, outputs)`) would require defining an explicit `Input` layer object, but it produces an identical internal computational graph.
 - **Algorithmic & Math Justification:** Serves as the primary structural container that handles weight tracking, layer-to-layer connections, forward pass routing, and backward loss propagation.
-

Line 15: `layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),`

- **Underlying Mechanics & Arguments:** Constructs a 2D spatial convolution layer.
 - `32` : Allocates 32 separate, independent trainable filter matrices.
 - `(3, 3)` : Configures each kernel filter with a spatial dimension of $3 \times 3 \times C_{in}$.
 - `activation='relu'` : Applies the Rectified Linear Unit element-wise mathematical function ($f(x) = \max(0, x)$) immediately after computing the spatial sums.
 - `input_shape=(28, 28, 1)` : Registers the dimension signature of incoming data tensors. The layer automatically instantiates a kernel weight tensor of shape `(3, 3, 1, 32)` and a bias vector of length 32.
 - **Structural & Shape Transformation:** * *Input:* Expects an incoming 4D matrix of shape `(None, 28, 28, 1)` .
 - *Output:* A 4D tensor of shape `(None, 26, 26, 32)` . The spatial resolution drops because valid padding (`padding='valid'`) is applied by default: $H_{out} = H_{in} - K_h + 1 = 28 - 3 + 1 = 26$.
 - **Behavioral Alternatives:** Increasing the kernel size to `(5, 5)` reduces the output spatial shape to `(None, 24, 24, 32)` and increases the parameter count per filter. Changing padding to `padding='same'` pads the input boundaries with zeros to preserve the spatial resolution at 28×28 .
 - **Algorithmic & Math Justification:** The cross-correlation operations extract shift-invariant localized features from the images. The activation function introduces non-linearity, enabling the model to learn complex structural relationships beyond simple linear transformations.
-

Line 16: `layers.MaxPooling2D((2, 2)),`

- **Underlying Mechanics & Arguments:** Instantiates a spatial maximum pooling layer. The tuple parameter `(2, 2)` defines both the downsampling window size and the stride length. Mechanically, a 2×2 grid slides across the feature maps with a non-overlapping stride of 2,

extracting only the maximum numerical value within each window. This operation does not introduce any trainable parameters.

- **Structural & Shape Transformation:** * *Input:* (None, 26, 26, 32) feature map tensor from the preceding layer.
 - *Output:* (None, 13, 13, 32) . Spatial dimensions are downsampled by half: $\lfloor 26/2 \rfloor = 13$, while the 32 channel planes remain decoupled and unchanged.
 - **Behavioral Alternatives:** Changing the stride or window dimension to (3, 3) with valid padding would result in an output shape of (None, 8, 8, 32) , causing a more aggressive loss of spatial details.
 - **Algorithmic & Math Justification:** Reduces the spatial dimensionality of the feature maps, decreasing the computational workload for downstream layers. It also introduces translation invariance, helping the network detect features regardless of minor spatial shifts.
-

Line 17: `layers.Conv2D(64, (3, 3), activation='relu'),`

- **Underlying Mechanics & Arguments:** Adds a second 2D convolutional layer into the architecture stack. It allocates 64 independent kernel tensors, each with a spatial layout size of 3×3 . The channel depth of these kernels automatically scales to match the incoming feature maps ($C_{in} = 32$). The total number of trainable parameter weights for this layer is calculated as: $\text{Weights} = (3 \times 3 \times 32) \times 64 = 18,432$. Adding 64 bias terms brings the total parameter count to 18,496.
 - **Structural & Shape Transformation:** * *Input:* Tensor of shape (None, 13, 13, 32) .
 - *Output:* Tensor of shape (None, 11, 11, 64) . The spatial resolution is reduced ($13 - 3 + 1 = 11$), while the channel depth expands to 64.
 - **Behavioral Alternatives:** Omitting the ReLU activation function here reverts the layer to a series of linear matrix multiplications, preventing the network from learning high-level abstract representations.
 - **Algorithmic & Math Justification:** Expanding the channel depth allows the network to combine low-level edge detections into more complex, high-level structural features.
-

Line 18: `layers.MaxPooling2D((2, 2)),`

- **Underlying Mechanics & Arguments:** Implements a second max pooling operation with a 2×2 pool size and a stride of 2. It slides across the 11×11 feature maps, dropping lower values and keeping only the maximum active signal within each window.
- **Structural & Shape Transformation:** * *Input:* Incoming tensor of shape (None, 11, 11, 64) .

- **Output:** Tensor of shape `(None, 5, 5, 64)` . Due to floor truncation during downsampling with valid padding: $\lfloor 11/2 \rfloor = 5$.
 - **Behavioral Alternatives:** Using `AveragePooling2D` instead computes the mathematical mean of the window area, which can blur sharp feature boundaries and reduce classification performance on high-contrast datasets.
 - **Algorithmic & Math Justification:** Further refines the spatial features, discarding uninformative pixel details while preserving the strongest activations across all 64 feature channels.
-

Line 19: `layers.Conv2D(64, (3, 3), activation='relu'),`

- **Underlying Mechanics & Arguments:** Adds a final convolutional layer to the network. It instantiates 64 filters with dimensions of $3 \times 3 \times 64$. The total number of trainable weights for this layer is computed as: $\text{Weights} = (3 \times 3 \times 64) \times 64 = 36,864$ Including 64 bias terms brings the total parameter count to 36,928.
 - **Structural & Shape Transformation:** * **Input:** Tensor of shape `(None, 5, 5, 64)` .
 - **Output:** Tensor of shape `(None, 3, 3, 64)` . The spatial grid is reduced to 3×3 ($5 - 3 + 1 = 3$).
 - **Behavioral Alternatives:** Adding another `MaxPooling2D((2, 2))` layer after this step would shrink the spatial dimensions down to 1×1 , severely restricting the capacity of the feature extractor.
 - **Algorithmic & Math Justification:** Captures deep, highly abstract spatial patterns before the feature maps are flattened and passed to the dense classification layers.
-

Line 20: `layers.Flatten(),`

- **Underlying Mechanics & Arguments:** Flattens the multi-dimensional feature maps into a 1D vector. This is a structural reshape operation that does not alter values or introduce trainable weights; it re-indexes the memory strides into a single continuous dimension.
 - **Structural & Shape Transformation:** * **Input:** 4D tensor of shape `(None, 3, 3, 64)` .
 - **Output:** 2D matrix of shape `(None, 576)` , where the feature length is calculated as the product of the spatial and channel dimensions: $3 \times 3 \times 64 = 576$.
 - **Behavioral Alternatives:** Omitting the flattening step causes a structural mismatch with the downstream dense layers, raising a compilation or runtime exception because `Dense` layers expect a flat feature vector.
 - **Algorithmic & Math Justification:** Converts spatial features into a single vector format, enabling fully connected matrix multiplications in the downstream classification layers.
-

Line 21: `layers.Dense(64, activation='relu'),`

- **Underlying Mechanics & Arguments:** Instantiates a fully connected (dense) neural network layer. It performs a matrix multiplication between the incoming feature vector and a weight matrix W , then adds a bias vector b . $\text{Output} = \text{ReLU}(X \cdot W + b)$ The weight matrix shape is initialized as $(576, 64)$, yielding $576 \times 64 = 36,864$ parameters. Adding 64 bias terms results in 36,928 total trainable parameters.
 - **Structural & Shape Transformation:** * *Input:* Flattened 2D matrix of shape `(None, 576)` .
 - *Output:* Matrix of shape `(None, 64)` . The data type remains `float32` .
 - **Behavioral Alternatives:** Drastically reducing the unit count (e.g., to 4) creates an extreme information bottleneck that can drop performance, while increasing it to 4096 drastically raises the parameter count and risks overfitting.
 - **Algorithmic & Math Justification:** Performs non-linear combinations of the extracted features, mapping them into a dense feature space optimized for the final classification task.
-

Line 22: `layers.Dense(10, activation='softmax')`

- **Underlying Mechanics & Arguments:** This defines the final classification layer of the network. It outputs 10 distinct values representing the target categories, using a weight matrix of shape $(64, 10)$ and a bias vector of length 10 ($64 \times 10 + 10 = 650$ parameters). It applies the Softmax activation function, which normalizes the raw output logits into a valid probability distribution where all values are bounded between $[0, 1]$ and sum to 1: $\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$
 - **Structural & Shape Transformation:** * *Input:* Incoming matrix of shape `(None, 64)` .
 - *Output:* Matrix of shape `(None, 10)` , where each row contains 10 probability scores.
 - **Behavioral Alternatives:** Setting the output units to any value other than 10 causes a size mismatch error against the target label classes. Replacing softmax with a linear activation returns unnormalized raw scores, which breaks the assumption of a categorical probability distribution.
 - **Algorithmic & Math Justification:** Converts the network's internal representations into a clear probability distribution, allowing the model's predictions to be directly evaluated against the true categorical targets.
-

Line 23: `])`

- **Underlying Mechanics & Arguments:** Closes the instantiation array list for the Keras `Sequential` container model. This executes the internal network graph setup, links the execution steps between layers, checks the structural shapes, and builds the tracking structures for all 93,322 parameters.

- **Structural & Shape Transformation:** Generates an active, compiled, non-trained Keras model graph instance.
 - **Behavioral Alternatives:** Syntactic closure. Syntax errors here will prevent parsing and execution.
 - **Algorithmic & Math Justification:** Completes the model's structural construction phase, wrapping all components into a single object for training and optimization.
-

Line 24: `model.compile(`

- **Underlying Mechanics & Arguments:** Invokes the Keras engine configuration routine to set up the optimizer, loss function, and tracking metrics. This establishes the numerical routines used during training, allocates memory for the optimization state vectors, and prepares the computational graph for high-performance execution.
 - **Structural & Shape Transformation:** Modifies the model's internal status flags from uncompiled to operational. The parameter shapes remain unchanged.
 - **Behavioral Alternatives:** Skipping this compilation step and calling `.fit()` immediately raises a runtime error, as the model cannot execute a backward pass without a defined optimizer and loss function.
 - **Algorithmic & Math Justification:** Configures the optimization target and updates rules, which are required before the model can compute gradients and update its weights during training.
-

Line 25: `optimizer='adam',`

- **Underlying Mechanics & Arguments:** Specifies the Adam (Adaptive Moment Estimation) optimization algorithm. Adam tracks the first moment (the mean) and the second moment (the uncentered variance) of the gradients using exponential moving averages. It computes individual adaptive learning rates for each parameter based on these moving averages, scaling the global learning rate α (defaulting to 0.001) dynamically during training: $\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$
 - **Structural & Shape Transformation:** Dynamically allocates auxiliary tracking tensors for each parameter to store first (m_t) and second (v_t) moment states.
 - **Behavioral Alternatives:** Switching to standard Stochastic Gradient Descent (`optimizer='sgd'`) drops the adaptive momentum tracking, which can lead to slower training convergence or cause the model to get stuck in local minima.
 - **Algorithmic & Math Justification:** Adam combines the benefits of AdaGrad and RMSProp, handling noisy gradients and complex loss landscapes efficiently without requiring extensive manual hyperparameter tuning.
-

Line 26: `loss='sparse_categorical_crossentropy',`

- **Underlying Mechanics & Arguments:** Specifies the loss function to minimize during training. Sparse Categorical Cross-Entropy is designed for multi-class classification tasks where target labels are provided as integers rather than one-hot encoded vectors. It calculates the negative log-likelihood of the true class probability: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{i,y_i})$ where y_i is the true class index and $\hat{y}$ is the predicted probability distribution.
 - **Structural & Shape Transformation:** No shape changes occur. It sets up an internal mathematical step that converts integer targets into indices to evaluate the softmax outputs.
 - **Behavioral Alternatives:** Using standard `categorical_crossentropy` instead requires converting the target labels into one-hot encoded vectors of shape `(None, 10)`, otherwise a shape mismatch exception will be raised during training.
 - **Algorithmic & Math Justification:** Quantifies the difference between predicted probabilities and actual class targets, producing smooth gradients that drive structural updates across the network.
-

Line 27: `metrics=['accuracy']`

- **Underlying Mechanics & Arguments:** Registers evaluation metrics to monitor during training and validation. The `'accuracy'` string tells Keras to calculate the ratio of correctly predicted samples to total samples by checking if the predicted class index matches the true label: $\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Samples}}$ This metric is computed for logging purposes and does not affect gradient updates.
 - **Structural & Shape Transformation:** Creates internal evaluation trackers. No modifications are made to the model's weight matrices.
 - **Behavioral Alternatives:** Omitting this parameter hides accuracy metrics during training logs, leaving only the raw loss values to monitor performance.
 - **Algorithmic & Math Justification:** Provides an easily interpretable performance metric to help evaluate model classification performance throughout the training process.
-

Line 28: `)`

- **Underlying Mechanics & Arguments:** Finalizes the `.compile()` configuration call, linking the optimizer and loss function tracking to the network's layers.
 - **Structural & Shape Transformation:** The network topology is now fully compiled and ready to receive data batches.
 - **Behavioral Alternatives:** Syntactic closure.
 - **Algorithmic & Math Justification:** Finalizes the compilation process, moving the network from setup to an executable state.
-

Line 29: `model.summary()`

- **Underlying Mechanics & Arguments:** Parses the network architecture graph and prints a summary to the console. It details each layer's type, output shape dimensions, and parameter counts, providing a clear overview of the model's structural design.
 - **Structural & Shape Transformation:** Text-only diagnostics. No changes are made to the internal model variables.
 - **Behavioral Alternatives:** Removing this statement has no impact on training, but it removes a valuable diagnostic check for verifying layer connections and shapes.
 - **Algorithmic & Math Justification:** Used to verify that layer transformations and feature shapes match the intended model design before starting training.
-

Code Cell 3: Training Loop and Evaluation Protocols

Line 30: `history = model.fit()`

- **Underlying Mechanics & Arguments:** Launches the main training loop execution environment. This method manages data batching, feeds batches through the forward pass, computes loss, performs backpropagation to calculate gradients, and updates weights across epochs. It returns a `History` object containing loss and metric logs recorded at each training step.
 - **Structural & Shape Transformation:** Dynamically mutates weight values across all trainable parameter tensors (totaling 93,322 parameters). It does not change tensor dimensions.
 - **Behavioral Alternatives:** Calling `.fit()` with unaligned or mismatched input arrays triggers an immediate structural shape mismatch error.
 - **Algorithmic & Math Justification:** Coordinates the optimization process, iteratively updating the model's weights to minimize the cross-entropy loss across the training dataset.
-

Line 31: `train_images,`

- **Underlying Mechanics & Arguments:** Passes the training image features as the primary input tensor. By default, Keras divides this dataset into mini-batches of 32 samples each during training.
- **Structural & Shape Transformation:** The input tensor is split into mini-batches of shape `(32, 28, 28, 1)` during each training step.
- **Behavioral Alternatives:** Providing incorrect tensor structures (such as a 3D tensor of shape `(60000, 28, 28)`) will cause the input layer to reject the data due to a dimension mismatch.

- **Algorithmic & Math Justification:** Supplies the raw feature arrays used to compute activations and feature representations across the network's layers.
-

Line 32: `train_labels,`

- **Underlying Mechanics & Arguments:** Supplies the target class labels corresponding to the training images, allowing the loss function to evaluate predictions and compute error gradients.
 - **Structural & Shape Transformation:** Mapped into mini-batches of shape `(32,)` during training steps to match the input image batches.
 - **Behavioral Alternatives:** If label indices do not match the expected categorical range (0 to 9), the loss function will fail with an out-of-bounds index error.
 - **Algorithmic & Math Justification:** Provides the ground-truth targets required to compute categorical cross-entropy loss and drive the optimization process.
-

Line 33: `epochs=10,`

- **Underlying Mechanics & Arguments:** Configures the model to run through the entire training dataset 10 times. One epoch means every training sample has passed through the forward and backward propagation loops exactly once.
 - **Structural & Shape Transformation:** The training loop runs for 10 complete iterations, updating weight parameters $10 \times (60000/32) = 18,750$ times.
 - **Behavioral Alternatives:** Setting `epochs=1` may lead to underfitting as the network lacks iterations to converge, while setting `epochs=1000` without regularization can cause severe overfitting.
 - **Algorithmic & Math Justification:** Controls training duration, ensuring the optimization algorithm has enough iterations to find a stable local minimum in the loss landscape.
-

Line 34: `validation_data=(test_images, test_labels)`

- **Underlying Mechanics & Arguments:** Passes the validation dataset as a tuple of features and labels. At the end of each epoch, the model pauses training and runs a forward pass over these evaluation samples to calculate loss and accuracy. This step does not compute gradients or update model weights.
- **Structural & Shape Transformation:** Processes evaluation batches of shape `(313, 28, 28, 1)` and labels of shape `(313,)` to compute validation metrics at the end of each epoch.
- **Behavioral Alternatives:** Omitting this parameter prevents tracking validation metrics during training, making it difficult to detect overfitting until training is complete.

- **Algorithmic & Math Justification:** Provides an independent performance benchmark on unseen data, allowing developers to monitor generalization and check for overfitting during training.
-

Line 35:)

- **Underlying Mechanics & Arguments:** Syntactic closure for the `.fit()` method call. This triggers the execution loop, printing epoch logs and training metrics to the console output.
 - **Structural & Shape Transformation:** Finalizes the training execution. The `history` object is populated with performance metrics across all 10 epochs.
 - **Behavioral Alternatives:** Syntactic closure.
 - **Algorithmic & Math Justification:** Marks the end of the training configuration block, initiating the model optimization process.
-

Line 36: `test_loss, test_acc = model.evaluate(`

- **Underlying Mechanics & Arguments:** Calls the model evaluation function to test performance on an independent dataset. It executes a single forward pass over the provided samples, calculates the average loss, and computes the final evaluation accuracy.
 - **Structural & Shape Transformation:** Unpacks the evaluation results into two scalar floating-point variables: `test_loss` and `test_acc`. The model's internal weights are not updated.
 - **Behavioral Alternatives:** If evaluation data has not undergone identical preprocessing (e.g., if pixel values remain unscaled between 0 and 255), the resulting evaluation metrics will show a significant drop in accuracy.
 - **Algorithmic & Math Justification:** Computes the final performance metrics on a holdout test set to evaluate how well the trained model generalizes to new data.
-

Line 37: `test_images,`

- **Underlying Mechanics & Arguments:** Passes the validation feature tensor to evaluate generalization performance.
 - **Structural & Shape Transformation:** Fed into the evaluation pipeline using the baseline test shape of `(10000, 28, 28, 1)`.
 - **Behavioral Alternatives:** Passing different features will evaluate performance against that specific data distribution instead of the intended test set.
 - **Algorithmic & Math Justification:** Supplies the unseen feature matrices required to benchmark the model's actual predictive capability.
-

Line 38: `test_labels,`

- **Underlying Mechanics & Arguments:** Passes the target labels for the test dataset, allowing the evaluation loop to verify predictions.
 - **Structural & Shape Transformation:** Array structure remains fixed at `(10000,)` .
 - **Behavioral Alternatives:** Swapping or mismatching the label array will cause the evaluation metrics to be calculated incorrectly against the wrong targets.
 - **Algorithmic & Math Justification:** Provides the true targets needed to accurately compute validation error and final model accuracy.
-

Line 39: `verbose=2`

- **Underlying Mechanics & Arguments:** Configures the logging output level during evaluation. A setting of `2` outputs a single summary line for the entire evaluation process, suppressing the step-by-step progress bars.
 - **Structural & Shape Transformation:** Changes console log formatting without altering any data structures or model variables.
 - **Behavioral Alternatives:** Setting `verbose=0` completely silences evaluation outputs, while `verbose=1` displays a real-time tracking progress bar for each batch.
 - **Algorithmic & Math Justification:** Adjusts console logging detail to maintain clean, readable output logs in automated testing pipelines.
-

Line 40: `)`

- **Underlying Mechanics & Arguments:** Finalizes the `.evaluate()` method call, completing the verification pass over the test dataset.
 - **Structural & Shape Transformation:** Completes evaluation execution and assigns values to `test_loss` and `test_acc` .
 - **Behavioral Alternatives:** Syntactic closure.
 - **Algorithmic & Math Justification:** Concludes the evaluation step, saving the resulting performance metrics for downstream tracking.
-

Line 41: `print(f"\nTest accuracy: {test_acc:.4f}")`

- **Underlying Mechanics & Arguments:** Constructs a formatted string containing the test accuracy rounded to four decimal places, then outputs it to the console.

- **Structural & Shape Transformation:** Displays diagnostic text without modifying any model or dataset tensors.
 - **Behavioral Alternatives:** Deleting this line has no impact on model behavior; it simply omits the final performance metric summary from the console log.
 - **Algorithmic & Math Justification:** Outputs a clean, human-readable summary of the model's final evaluation accuracy on unseen test data.
-

Code Cell 4: Prediction Pipeline & Statistical Visualizations

Line 42: `predictions = model.predict(test_images)`

- **Underlying Mechanics & Arguments:** Runs a forward inference pass over the evaluation dataset. It passes the inputs through the entire network layer stack, applying the learned weights and activations to compute a final probability distribution for each sample via the softmax output layer.
 - **Structural & Shape Transformation:** * *Input:* `test_images` tensor of shape `(10000, 28, 28, 1)` .
 - *Output:* A 2D NumPy float matrix named `predictions` with a shape of `(10000, 10)` , where each row contains 10 class probabilities that sum to 1.
 - **Behavioral Alternatives:** Modifying or shuffling the input array structure will disrupt the spatial alignment, causing the inference pass to fail with dimension errors.
 - **Algorithmic & Math Justification:** Generates the raw predictive probability distributions used for downstream analysis, error checking, and data visualization.
-

Line 43: `def plot_image(i, predictions_array, true_label, img):`

- **Underlying Mechanics & Arguments:** Declares a custom plotting function designed to visualize an image along with its predicted class probability, true label, and a color-coded status indicator.
 - **Structural & Shape Transformation:** Registers a function reference in the local namespace workspace without altering any data arrays.
 - **Behavioral Alternatives:** If this signature is missing required inputs or misconfigured, subsequent visualization calls will raise a standard `TypeError` .
 - **Algorithmic & Math Justification:** Packages visualization logic into a reusable block, making it easy to generate consistent diagnostic plots across different image batches.
-

Line 44: `true_label, img = true_label[i], img[i].reshape(28, 28)`

- **Underlying Mechanics & Arguments:** Extracts the specific sample data for index `i` . It selects the true integer class label and slices the corresponding image array out of the 4D input tensor, flattening the single-channel spatial dimensions from `(28, 28, 1)` to a 2D matrix shape of `(28, 28)` .
 - **Structural & Shape Transformation:** * *Input:* Arrays `true_label` of shape `(10000,)` and `img` of shape `(10000, 28, 28, 1)` .
 - *Output:* A scalar integer assigned to `true_label` , and a 2D matrix assigned to `img` with a shape of `(28, 28)` .
 - **Behavioral Alternatives:** Removing the `.reshape(28, 28)` operation leaves the image as a 3D tensor of shape `(28, 28, 1)` . While `plt.imshow` can process this structure, explicit flattening prevents channel-handling errors across different visualization backends.
 - **Algorithmic & Math Justification:** Isolates and formats a single image sample, converting it to the 2D matrix format required by standard plotting libraries.
-

Line 45: `plt.grid(False)`

- **Underlying Mechanics & Arguments:** Disables the overlay grid lines on the active Matplotlib axis canvas, ensuring a clean background for image rendering.
 - **Structural & Shape Transformation:** Modifies target axis visualization properties without changing underlying image values.
 - **Behavioral Alternatives:** Removing this line may cause grid lines to display over the image depending on default style settings, which can obscure fine visual details.
 - **Algorithmic & Math Justification:** Removes uninformative visual elements from the canvas to ensure a clear view of the target image features.
-

Line 46: `plt.xticks([])`

- **Underlying Mechanics & Arguments:** Passes an empty list to clear all tick marks and numeric labels from the horizontal X-axis line.
 - **Structural & Shape Transformation:** Disables X-axis ticks on the active sub-plot axis.
 - **Behavioral Alternatives:** Leaving this setting at default values prints raw pixel coordinate indices (0 to 28) along the bottom edge, which adds visual clutter to the diagnostic grid.
 - **Algorithmic & Math Justification:** Removes structural pixel coordinate labels, keeping the layout clean and focused on the visual features.
-

Line 47: `plt.yticks([])`

- **Underlying Mechanics & Arguments:** Clears all tick marks and numeric labels from the vertical Y-axis line by passing an empty list parameter.
 - **Structural & Shape Transformation:** Removes vertical axis markings from the active canvas area.
 - **Behavioral Alternatives:** Default behaviors print vertical pixel indices (0 to 28) along the left edge, which can clutter compact multi-plot layouts.
 - **Algorithmic & Math Justification:** Cleans up the vertical axis display to optimize the visual presentation of the image grid.
-

Line 48: `plt.imshow(img, cmap=plt.cm.binary)`

- **Underlying Mechanics & Arguments:** Renders the 2D pixel matrix onto the active axis canvas. The `cmap=plt.cm.binary` parameter applies a binary colormap that displays zero values as white and maximum intensities as black, matching standard grayscale visualization conventions.
 - **Structural & Shape Transformation:** Maps the values of the 28×28 matrix into a color-rendered image plot on the canvas.
 - **Behavioral Alternatives:** Omitting the colormap argument uses Matplotlib's default colormap (typically viridis), which displays grayscale images in false-color purple and yellow tones.
 - **Algorithmic & Math Justification:** Maps raw numeric pixel intensities back into a standard visual representation, allowing for manual verification of image features.
-

Line 49: `predicted_label = np.argmax(predictions_array)`

- **Underlying Mechanics & Arguments:** Invokes the NumPy `np.argmax()` function to identify the index of the highest probability value within the 1D predictions array, extracting the network's top prediction for that sample: $\text{Index} = \arg \max_j (\mathbf{p}_j)$
 - **Structural & Shape Transformation:** * *Input:* A 1D array of 10 probability scores.
 - *Output:* A single scalar integer between 0 and 9 assigned to `predicted_label` .
 - **Behavioral Alternatives:** Using a min reduction (`np.argmin`) would extract the class the model considers least likely, completely reversing the evaluation logic.
 - **Algorithmic & Math Justification:** Converts the continuous probability distribution output by the softmax layer into a single categorical class prediction for direct evaluation.
-

Line 50: `color = 'blue' if predicted_label == true_label else 'red'`

- **Underlying Mechanics & Arguments:** Executes a conditional inline expression that sets a text color variable based on prediction accuracy. If the predicted index matches the ground truth label, it returns 'blue' ; if they differ, it returns 'red' .
 - **Structural & Shape Transformation:** Assigns a text string configuration value to the color variable. No data structures are altered.
 - **Behavioral Alternatives:** Using static colors removes the visual indicator for prediction errors, requiring manual inspection of text labels to catch misclassifications.
 - **Algorithmic & Math Justification:** Provides an immediate visual cue for evaluation analysis, making it easy to spot classification errors in large image grids.
-

Line 51: `plt.xlabel(`

- **Underlying Mechanics & Arguments:** Sets a text label along the horizontal axis of the sub-plot, allowing for dynamic formatting of the prediction text, confidence score, and true label.
 - **Structural & Shape Transformation:** Appends a text string label to the X-axis of the active canvas area.
 - **Behavioral Alternatives:** Missing or malformed parameters will trigger a string parsing or layout formatting error during rendering.
 - **Algorithmic & Math Justification:** Adds detailed textual labels to the plot, making it easy to audit the model's predictions directly alongside the image sample.
-

Line 52: `f"{class_names[predicted_label]} {100*np.max(predictions_array):.2f}% "`

- **Underlying Mechanics & Arguments:** Formats the first part of the label string. It looks up the text name for the predicted class index in `class_names` , extracts the highest probability value using `np.max()` , multiplies it by 100, and formats it as a percentage string rounded to two decimal places.
 - **Structural & Shape Transformation:** Computes a temporary text string snippet within the f-string evaluation scope.
 - **Behavioral Alternatives:** Using raw formatting fields without limiting decimal precision (e.g., omitting `:.2f`) produces long text strings that can overlap and clutter the visual layout.
 - **Algorithmic & Math Justification:** Displays the model's prediction confidence alongside the predicted category name, helping evaluate the strength of its classifications.
-

Line 53: `f"({class_names[true_label]})"`,

- **Underlying Mechanics & Arguments:** Evaluates the second part of the f-string, looking up the actual ground-truth class name from the tracking array and wrapping it in parentheses for clear visual separation.
 - **Structural & Shape Transformation:** Appends the true category name to the formatted text layout string.
 - **Behavioral Alternatives:** Hardcoding index integers instead of looking up names forces the viewer to manually decode class numbers back to category terms during review.
 - **Algorithmic & Math Justification:** Includes the ground truth label in the visualization, allowing for quick confirmation of whether a prediction is correct or incorrect.
-

Line 54: `color=color`

- **Underlying Mechanics & Arguments:** Passes the color-coding string variable (`'blue'` or `'red'`) to the text rendering configuration parameter.
 - **Structural & Shape Transformation:** Changes the text color property on the plot layout canvas.
 - **Behavioral Alternatives:** Setting a fixed color value disables the clear visual distinction between correct and incorrect predictions.
 - **Algorithmic & Math Justification:** Integrates the error-dependent color coding into the label display, making it easier to parse results during diagnostic checks.
-

Line 55: `)`

- **Underlying Mechanics & Arguments:** Closes the `plt.xlabel` function call, saving the formatted text configuration to the current sub-plot axis properties.
 - **Structural & Shape Transformation:** Completes the layout setup for the active sub-plot axis.
 - **Behavioral Alternatives:** Syntactic closure.
 - **Algorithmic & Math Justification:** Finalizes the label configuration block, applying the formatted text to the image axis.
-

Line 56: `num_rows = 5`

- **Underlying Mechanics & Arguments:** Defines an integer variable specifying the number of horizontal rows to include in the final visualization grid layout.
- **Structural & Shape Transformation:** Allocates an integer primitive variable named `num_rows` in memory.
- **Behavioral Alternatives:** Changing this value alters the total number of images displayed on the evaluation canvas.

- **Algorithmic & Math Justification:** Sets a core layout variable used to calculate sub-plot grid spacing and canvas size adjustments.
-

Line 57: `num_cols = 3`

- **Underlying Mechanics & Arguments:** Assigns an integer variable defining the number of vertical columns for the visualization grid layout.
 - **Structural & Shape Transformation:** Registers an integer primitive variable named `num_cols` in memory.
 - **Behavioral Alternatives:** Increasing the column count creates a wider visualization grid, which may require resizing the figure window to avoid squeezed plots.
 - **Algorithmic & Math Justification:** Works alongside the row configuration variable to manage grid positioning across the final visualization canvas.
-

Line 58: `num_images = num_rows * num_cols`

- **Underlying Mechanics & Arguments:** Multiplies the row and column counts to calculate the total number of images to display in the plot grid loop: $5 \times 3 = 15$.
 - **Structural & Shape Transformation:** Instantiates an integer variable named `num_images` with a calculated value of 15.
 - **Behavioral Alternatives:** Manually hardcoding this value instead of calculating it dynamically can lead to indexing discrepancies if row or column dimensions are modified later.
 - **Algorithmic & Math Justification:** Establishes the loop boundaries required to extract and plot the exact number of targeted image samples.
-

Line 59: `plt.figure(figsize=(2 * 2 * num_cols, 2 * num_rows))`

- **Underlying Mechanics & Arguments:** Creates a new top-level figure window container using the `figsize` parameter to specify the width and height dimensions in inches. For this configuration, it computes dimensions of: $\text{Width} = 2 \times 2 \times 3 = 12$ inches, $\text{Height} = 2 \times 5 = 10$ inches
 - **Structural & Shape Transformation:** Allocates a clean display figure canvas matching the calculated dimensions (12×10 inches).
 - **Behavioral Alternatives:** Omitting explicit sizing values falls back to standard Matplotlib defaults (6.4×4.8 inches), which can crush and distort a dense 15-plot layout.
 - **Algorithmic & Math Justification:** Allocates sufficient display canvas area to prevent text labels and sub-plots from overlapping in multi-grid visualizations.
-

Line 60: `for i in range(num_images):`

- **Underlying Mechanics & Arguments:** Initializes a standard loop that iterates through index values from 0 up to `num_images - 1` (14). This drives the sequential generation of each sub-plot in the grid.
 - **Structural & Shape Transformation:** Controls execution tracking, updating the loop index variable `i` across 15 sequential iterations.
 - **Behavioral Alternatives:** Iterating beyond the available range of the test dataset will raise an out-of-bounds `IndexError` during image extraction steps.
 - **Algorithmic & Math Justification:** Automates the generation of the sub-plot layout, ensuring each selected image sample is processed and positioned correctly.
-

Line 61: `plt.subplot(num_rows, 2 * num_cols, 2 * i + 1)`

- **Underlying Mechanics & Arguments:** Creates a specific sub-plot axis within the active figure canvas. The layout grid is configured with `num_rows` (5) and `2 * num_cols` (6) positions, creating a total of 30 available slots. The final parameter determines the active position for the current iteration, using odd slots (1, 3, 5, ...) to leave adjacent spaces clear for additional metrics or clean padding.
 - **Structural & Shape Transformation:** Allocates a target sub-plot axis structure at the calculated grid index position on the canvas.
 - **Behavioral Alternatives:** Passing an index value outside the allocated grid dimensions (1 to 30) will cause the plotting pipeline to fail with a structural value error.
 - **Algorithmic & Math Justification:** Manages layout positioning across the multi-column display grid, ensuring each image plot is assigned to its correct coordinates.
-

Line 62: `plot_image(i, predictions[i], test_labels, test_images)`

- **Underlying Mechanics & Arguments:** Calls the custom plotting function defined in line 43. It passes the current loop index `i`, the prediction probability vector for that sample, the true label array, and the full test image array to render the data onto the active sub-plot axis.
- **Structural & Shape Transformation:** Generates the complete image visualization, adding the rendered image, color-coded text labels, and prediction confidence metrics to the active sub-plot axis.
- **Behavioral Alternatives:** Passing mismatched array indexes will link images with unrelated predictions and ground-truth targets, producing incorrect diagnostic plots.
- **Algorithmic & Math Justification:** Executes the plotting logic for each sample, combining predictions and ground truth into a single coherent visualization for easy model auditing.

Line 63: `plt.tight_layout()`

- **Underlying Mechanics & Arguments:** Invokes Matplotlib's layout engine to automatically adjust sub-plot parameters. It computes padding offsets and resizes plot boundaries to ensure that labels, titles, and axes do not overlap across the grid.
 - **Structural & Shape Transformation:** Updates layout coordinates across all sub-plots on the figure canvas to optimize spacing.
 - **Behavioral Alternatives:** Removing this call leaves layout spacing at default values, which often causes text labels to overlap with adjacent sub-plots in dense grids.
 - **Algorithmic & Math Justification:** Optimizes plot alignment and spacing, ensuring that all text labels and visual features are clean and legible.
-

Line 64: `plt.show()`

- **Underlying Mechanics & Arguments:** Finalizes the visualization process. It flushes the rendering buffers, activates the display backend interface, and renders the completed figure canvas onto the screen or user interface. Once displayed, it clears the state stack to prepare for future plotting operations.
- **Structural & Shape Transformation:** Outputs the final evaluation grid to the interface window and resets the internal Matplotlib canvas tracking state.
- **Behavioral Alternatives:** In non-interactive or headless environments, calling `.show()` can execute without displaying an output window; in these setups, `plt.savefig('output.png')` is used instead to write the canvas directly to disk.
- **Algorithmic & Math Justification:** Displays the complete diagnostic visualization grid, providing a clear overview of the model's classification performance across a sample of test images.